



Regalia

A CARES edition for the Knights of Columbus

TECHNICAL REFERENCE • AUDIT & CRYPTOGRAPHY

How the audit chain actually works

Four independent guarantees — capture, attribution, tamper-evidence, and tamper-resistance — implemented in a PostgreSQL trigger, a SHA-256 hash chain, a transaction-scoped advisory lock, and a set of table grants. This document sets out the mechanism, the code that implements it, and the boundaries of each claim.

DOCUMENT	Audit chain technical reference
SYSTEM	CARES — Community Accounting & Resource Engagement System
APPLIES TO	Section 145 semi-annual trustee audit
SOURCES	audit_schema.py, audit_context.py, audit_routes.py, models.py
ISSUED	August 28, 2026



CONTENTS

1. Four guarantees, four mechanisms

2. Capture: the trigger, not the application

3. Attribution: a person's name in a trigger

4. Tamper-evidence: the hash chain
5. Verification: two queries, two attacks

6. Tamper-resistance: the grants

7. Limits: what this does not protect

8. In use: the Trustee Audit Report

1 · THE CLAIM, PRECISELY

Four guarantees, four different mechanisms

These are routinely collapsed into “we have an audit log.” They are not the same property, they fail independently, and only two of them are cryptographic. Distinguishing them is the whole point of the design.

PROPERTY	MECHANISM	WHAT IT GIVES YOU	WHAT IT DOES NOT
Capture	Row-level <code>AFTER</code> trigger on 20 tables	Every write is recorded regardless of code path — app, script, or raw <code>psql</code>	Says nothing about whether the record survives
Attribution	<code>SET LOCAL app.current_user_id</code> read back inside the trigger	Ties a change to an application user, not just a database role	Trusts the app to set it; a direct SQL writer is simply <code>NULL</code>
Tamper-evidence	SHA-256 chain, each row hashing the previous row’s hash	Any edit, deletion, or insertion is <i>detectable</i> by anyone with read access	Does not prevent it; unkeyed, so a privileged actor can re-forged forward
Tamper-resistance	PostgreSQL grants: <code>SELECT, INSERT</code> only on <code>audit_log</code>	The application’s own role <i>cannot</i> rewrite history, even via SQL injection	Binds the app role only — table owner and superuser are unaffected

2 · CAPTURE

The trigger, not the application

Every audited table carries the same row-level trigger. `install_audit_triggers()` attaches it by iterating `AUDITED_TABLES` — 20 tables covering the ledger (`journal_entries`, `journal_entry_lines`), membership (`members`, `membership_events`, `member_dues_payments`), access control (`users`, `organizations`), and the Form 1295 submission record.

```

connection.execute(text("CREATE EXTENSION IF NOT EXISTS pgcrypto"))
connection.execute(text(_TRIGGER_FUNCTION_SQL))
for table in AUDITED_TABLES:
    connection.execute(text(f"DROP TRIGGER IF EXISTS trg_audit_{table} ON {table}"))
    connection.execute(text(
        f"CREATE TRIGGER trg_audit_{table} "
        f"AFTER INSERT OR UPDATE OR DELETE ON {table} "
        f"FOR EACH ROW EXECUTE FUNCTION audit_trigger_fn()"
    ))

```

```
audit_schema.py :: install_audit_triggers()
```

The placement is the argument. An application-level hook — a SQLAlchemy event, a decorator on a service function — is bypassed by exactly the person the log exists to catch: someone with database credentials and a reason to edit the books quietly. A trigger fires beneath all of that. A treasurer who opens `psql` and runs `UPDATE journal_entry_lines SET debit_amount = 0 WHERE id = 8814` produces an audit row with the full before and after state, and no application code was involved in producing it.

The captured state is the whole row, both sides, as `JSONB`:

```

IF TG_OP = 'DELETE' THEN
    v_old := to_jsonb(OLD);
    v_new := NULL;
    v_row_id := (v_old->>'id')::INTEGER;
ELSIF TG_OP = 'UPDATE' THEN
    v_old := to_jsonb(OLD);
    v_new := to_jsonb(NEW);
    v_row_id := (v_new->>'id')::INTEGER;
ELSE
    v_old := NULL;
    v_new := to_jsonb(NEW);
    v_row_id := (v_new->>'id')::INTEGER;
END IF;

```

```
audit_schema.py :: audit_trigger_fn()
```

Because the trigger is the only writer, the ORM model is defended against becoming a second one. `AuditLog` exists so the report can read the table; three SQLAlchemy listeners ensure nothing ever writes through it:

```
def _forbid_orm_writes_to_audit_log(mapper, connection, target):
    raise RuntimeError(
        "AuditLog rows must never be written through the ORM -- only "
        "audit_trigger_fn() (see audit_schema.py) writes this table. ..."
    )

sa_event.listen(AuditLog, 'before_insert', _forbid_orm_writes_to_audit_log)
sa_event.listen(AuditLog, 'before_update', _forbid_orm_writes_to_audit_log)
sa_event.listen(AuditLog, 'before_delete', _forbid_orm_writes_to_audit_log)
```

models.py

3 · ATTRIBUTION

Getting a person's name into a database trigger

The trigger sees one thing about identity: `current_user`, the PostgreSQL role the whole application connects as. Every change would be attributed to `regalia_app`, which is useless to a trustee. The application therefore announces the acting user to the database once per transaction, in a setting the trigger reads back.

```
_current_actor_id = contextvars.ContextVar('cares_current_actor_id', default=None)

@event.listens_for(Session, "after_begin")
def _apply_actor_to_transaction(session, transaction, connection):
    actor_id = _current_actor_id.get()
    connection.execute(
        text("SET LOCAL app.current_user_id = :actor_id"),
        {'actor_id': str(actor_id) if actor_id is not None else ''},
    )
```

services/audit_context.py

Three details carry weight. The listener is registered on the SQLAlchemy `Session` class, not an instance, so it applies to every session in the process — including the test harness's independently constructed `sessionmaker`, which never imports this module. `SET LOCAL` is transaction-scoped, so the value dies at commit or rollback and cannot leak into an unrelated unit of work. And the actor lives in a `contextvars.ContextVar`, set in `before_request` and cleared in `teardown_request`, so a recycled worker thread does not inherit the previous request's identity.

The trigger reads it defensively, because the setting may not exist at all:

```

BEGIN
    v_user_id := NULLIF(current_setting('app.current_user_id', true), '')::INTEGER;
EXCEPTION WHEN others THEN
    v_user_id := NULL;
END;

```

audit_schema.py :: audit_trigger_fn()

A NULL ACTOR IS A FINDING, NOT A GAP

`changed_by_user_id IS NULL` means the change did not come from a logged-in request. On a development box that is a maintenance script. On a council's production system it means someone changed the books outside the application, and it is precisely the row a trustee should ask about. The `db_role` column records `current_user` alongside it, so a change made by a role other than the application's own is visible on its face.

A real bug this design produced

`after_begin` fires when a transaction *begins* — not retroactively when the actor changes mid-transaction. The login route originally called `set_current_actor(user.id)` *after* its `User.query` lookup had already opened the request's transaction, so the `last_login` update was attributed to nobody. The fix was to commit immediately after the lookup, so the write lands in a fresh transaction that already carries the correct actor. Worth knowing before writing any new code that sets the actor late.

4 · TAMPER-EVIDENCE

The hash chain

Each row's `row_hash` is a SHA-256 digest, hex-encoded to 64 characters, over a pipe-delimited payload that includes the *previous* row's hash. That single inclusion is what turns a list of records into a chain: the digest of row *n* depends transitively on the exact contents of every row before it.

```

SELECT row_hash INTO v_prev_hash FROM audit_log ORDER BY id DESC LIMIT 1;

v_payload := coalesce(v_prev_hash, '<genesis>') || '|' || TG_TABLE_NAME || '|' || TG_OP || '|'
            || coalesce(v_old::text, '') || '|' || coalesce(v_new::text, '')
            || '|' || coalesce(v_user_id::text, '<unknown>') || '|' || v_changed_at::text;

v_row_hash := encode(digest(v_payload, 'sha256'), 'hex');

```

audit_schema.py :: audit_trigger_fn()

The payload commits to seven fields: the previous hash, the table, the operation, the complete before state, the complete after state, the acting user, and the timestamp. Nothing a forensic reader would care about is left outside

the digest. The sentinels `<genesis>` and `<unknown>` stand in for a null previous hash and a null actor, so those cases hash deterministically rather than collapsing to an empty string another row could also produce.

WHY THE TIMESTAMP IS CAPTURED INTO A VARIABLE FIRST

`v_changed_at := clock_timestamp();` runs once, and the same value is used in both the hash payload and the stored column. `clock_timestamp()` is *volatile* — it returns the real wall clock at the moment of each call, not a value frozen for the transaction. Calling it separately in the payload and in the `INSERT` would store one timestamp and hash a slightly different one, and every single row would then fail verification. A one-line detail that decides whether the entire feature works.

Why one global advisory lock

The chain has a race condition built into its definition: `SELECT ... ORDER BY id DESC LIMIT 1` reads the current tail. Two concurrent transactions writing to two different audited tables would both read the same tail, and both would write a row claiming the same `prev_hash` — a fork, after which “the chain” is no longer a single sequence and verification cannot say which branch is authentic. So every audited write in the system, across every table, serializes behind one lock:

```
PERFORM pg_advisory_xact_lock(hashtext('cares_audit_log_chain'));
```

`audit_schema.py :: audit_trigger_fn(), first statement`

Transaction-scoped, so it releases at commit or rollback with no cleanup path to forget. The cost is real and worth stating plainly: all audited writes are serialized globally, and any long-running transaction touching an audited table holds the lock for its entire duration. At a single council’s transaction volume this is irrelevant. It would not be at a different scale.



Each row copies the previous row's `row_hash` into its own `prev_hash` and folds it into its digest. An edit invalidates the row's own hash and severs its link to the next row; a deletion leaves every surviving hash internally valid and still severs the link. The two verification queries below catch these separately, which is why both exist.

5 · VERIFICATION

Two queries, two different attacks

`POST /audit/verify` runs both checks over the entire table — not the date range currently on screen, because a break anywhere invalidates everything after it, including ranges that look untouched. The first recomputes every row's digest from that row's own stored columns and compares it to the stored hash:

```

SELECT id, table_name, operation, changed_at
FROM audit_log
WHERE row_hash <> encode(digest(
  coalesce(prev_hash, '<genesis>') || '|' || table_name || '|' || operation || '|' ||
  coalesce(old_data::text, '') || '|' || coalesce(new_data::text, '') || '|' ||
  coalesce(changed_by_user_id::text, '<unknown>') || '|' || changed_at::text,
  'sha256'), 'hex')
ORDER BY id

```

blueprints/audit_routes.py :: verify()

The second ignores content entirely and checks only linkage, using a window function to compare each row's declared predecessor against its actual one:

```

WITH ordered AS (
    SELECT id, table_name, operation, changed_at, prev_hash,
           lag(row_hash) OVER (ORDER BY id) AS expected_prev_hash
    FROM audit_log
)
SELECT id, table_name, operation, changed_at
FROM ordered
WHERE prev_hash IS DISTINCT FROM expected_prev_hash
ORDER BY id

```

```
blueprints/audit_routes.py :: verify()
```

Neither check subsumes the other. Editing a row's contents and recomputing only that row's hash passes check one and fails check two at the following row. Deleting a row entirely leaves every remaining row internally consistent — check one passes cleanly — and breaks check two at the deletion point. Note also that `IS DISTINCT FROM` rather than `<>` means the genesis row is validated too: its `prev_hash` must be `NULL`, and `lag()` over the first row is `NULL`, so a forged row prepended to the table is caught rather than skipped by null-comparison semantics.

THE VERIFICATION IS NOT PRIVILEGED CODE

Both queries are ordinary SQL run against ordinary columns, using the same `digest()` function from `pgcrypto` that wrote them. Anyone with `SELECT` on `audit_log` — a trustee, an outside accountant, an examiner — can run them without the application, without its source, and without trusting it. That independence is the property that makes the log evidence rather than a report.

6 · TAMPER-RESISTANCE

What the database refuses to let the application do

Hashes detect. Grants prevent. In production the application connects as a role that can run the business, write new audit rows through the trigger, and do nothing else to the log:

```

business_tables = ', '.join(AUDITED_TABLES + ['translation_cache', 'usage_events'])
connection.execute(text(f"GRANT SELECT, INSERT, UPDATE, DELETE ON {business_tables} TO
{role_ident}"))

connection.execute(text(f"GRANT SELECT, INSERT ON audit_log TO {role_ident}"))
connection.execute(text(f"REVOKE UPDATE, DELETE, TRUNCATE ON audit_log FROM {role_ident}"))

```

```
audit_schema.py :: grant_restricted_runtime_role()
```

The consequence is that a SQL-injection vulnerability anywhere in the application, or possession of the application's own database password, is not sufficient to alter audit history. The most such an actor can do is append — and appended rows chain onto the existing tail, leaving the earlier record intact and verifiable.

This function is deliberately *not* wired into any startup path. It is run once, by hand, by whoever controls the database, using an owner-level role. Automating it would mean the application's own credentials could re-grant themselves whatever they liked at boot, which is exactly the capability the grants exist to remove.

KNOWN OPERATIONAL GAP

Nothing currently verifies that a given deployment has actually applied these grants. An installation that skipped the step runs as an unrestricted role, keeps its *tamper-evidence*, and silently loses its *tamper-resistance*. A startup check confirming the app's own connection cannot `UPDATE audit_log` is an open item.

7 · LIMITS

What this does not protect against

The design is worth exactly what its weakest honest claim is worth, so here are the boundaries.

The hashes are unkeyed

This is a hash chain, not a signature scheme. There is no HMAC key and no private key anywhere in the system — `digest(payload, 'sha256')` is computable by anyone. Security comes from the append-only grants and from the impracticality of undetected rewriting, not from secrecy. An actor with owner or superuser rights on the database can edit a row and then recompute every subsequent `row_hash` and `prev_hash`, producing a chain that verifies cleanly. Nothing in the current design detects that.

There is no external anchor

The standard defence against a full-suffix rewrite is to publish the chain head somewhere the operator does not control — mailing the current head hash to trustees monthly, or committing it to an external system — so any rewrite contradicts a value already witnessed elsewhere. That is not implemented. It is the single highest-value addition available and it is cheap: one periodic message containing 64 hex characters would make a silent rewrite of history detectable by anyone holding an old copy.

It records intent-neutral facts

A logged-in treasurer entering a fraudulent but well-formed transaction produces a perfectly valid, correctly attributed, cryptographically sound audit row. The chain proves what was recorded and by whom; it does not judge whether the entry was honest. Detecting that is what the Form 1295 schedules, the reconciliation checks, and human trustees are for.

Verification cost grows without bound

`/audit/verify` scans the entire table on every run. For a single council this is trivial. There is no checkpointing of a last-verified id and no incremental mode, so the cost rises linearly with history forever.

Timestamps are server wall-clock

`clock_timestamp()` reflects the database server’s clock, which an administrator can move. The chain fixes the *order* of events beyond dispute; it does not independently establish the absolute time any of them occurred.

8 · IN USE

The Trustee Audit Report

Knights of Columbus Section 145 requires a council’s trustees to audit the financial secretary’s and treasurer’s books at least semi-annually, so `/audit/log` defaults to a six-month window and can be filtered by table. For `UPDATE` rows it renders a field-level diff computed from the stored `old_data` and `new_data` JSON, so a reviewer sees which columns changed rather than two blobs. Results cap at 500 rows with an explicit truncation notice, and a “Verify Chain Integrity” action runs both queries above.

One deliberate schema choice supports this: `changed_by_user_id` carries **no foreign key** to `users`. The actor is resolved with a plain lookup at render time, falling back to “System / Unknown”. A log whose rows could be invalidated by later deleting a user would not be a record of what happened.

PRACTICAL NOTE FOR OPERATORS

The triggers are installed by `init_db.py` and by the demo loader, but *not* by `migrate_production.py`. A database brought up only through the migration path has no audit trail at all, and the report will be empty regardless of activity. `install_audit_triggers()` is idempotent, but it reaches idempotency by dropping and recreating each trigger — which takes an `ACCESS EXCLUSIVE` lock — so it should be called when nothing else holds a transaction open against an audited table.

Sources: `audit_schema.py`, `services/audit_context.py`, `blueprints/audit_routes.py`, `models.py`. Code excerpts are quoted from the implementation; hex digests in the diagram are illustrative. Full deployment procedure for the restricted runtime role is in `AUDIT_TRAIL.md`.